

Classes Pile, FileAttente et FileAttentePr

Une *pile* est une liste où les insertions et les suppressions se font toutes du même côté.

Une *file d'attente* est une liste où les insertions se font d'un côté et les suppressions se font de l'autre côté.

Les piles et les files d'attente ont deux comportements en commun : `estVide` et `toString`. Nous définissons la classe abstraite `PileFile` afin de factoriser ces deux fonctionnalités.

```
public abstract class PileFile<E> {
    AbstractList<E> rep;
    public PileFile() {...}
    public boolean estVide() {...}
    public String toString() {...}
}

public class PileFileException
    extends RuntimeException{
    public PileFileException (String mess) {...}
}
```

Définir les méthodes de la classe `PileFile` et de la classe `PileFileException` :

- La méthode `estVide` retourne **true** si la pile-file est vide et **false** sinon.
- La méthode `toString` retourne une chaîne de caractères représentant le contenu de la pile : la liste des objets de la pile-file entre `[]` et séparés par des virgules.

Pile

Définir les méthodes de la classe `Pile` :

- Pour définir le constructeur, on choisira la représentation la plus « performante » en fonction des opérations effectuées sur la pile.
- La méthode *empiler* ajoute un objet en sommet de pile.
- La méthode *depiler* enlève l'objet se trouvant en sommet de pile et le retourne.
- La méthode *sommet* retourne l'objet en sommet de pile sans l'enlever de la pile.

```
public class Pile<E> extends PileFile<E>{
    public Pile () {...}
    public void empiler(E e) {...}
    public E depiler() {...}
    public E sommet() {...}
}
```

File d'attente

Définir les méthodes de la classe `FileAttente` :

- Pour définir le constructeur, on choisira la représentation la plus « performante » en fonction des opérations effectuées sur la file d'attente.
- La méthode *enfiler* ajoute un objet en fin de file.
- La méthode *defiler* enlève l'objet se trouvant en début de file d'attente et le

```
public class FileAttente<E> extends
PileFile<E> {
    public FileAttente() {...}
    public void enfiler(E e) {...}
    public E defiler() {...}
    public E premier() {...}
    public E dernier() {...}
}
```

retourne.

- La méthode `premier` retourne l'objet en début de file d'attente sans l'enlever de la file d'attente.
- La méthode `dernier` retourne l'objet en fin de file d'attente sans l'enlever de la file d'attente.

File d'attente avec priorité

Une file d'attente avec priorité est une file d'attente dans laquelle l'ordre suivant lequel les objets sont sortis dépend de leur priorité : `defiler` sort l'objet le plus prioritaire de la file d'attente. Les objets stockés dans une file d'attente avec priorité sont des objets qui possèdent une priorité : ils doivent implémenter l'interface `Priorisable`.

```
public interface Priorisable {
    int getPriorite();
}
```

Les comportements de la classe `FileAttentePr` sont les mêmes que les fonctionnalités de la classe `FileAttente`. Définir la classe `FileAttentePr`.

File d'attente avec priorité gérée par un «heap»

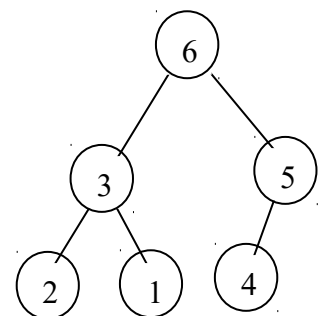
La file d'attente peut être représentée par un arbre binaire ordonné horizontalement : tout nœud de l'arbre binaire a une valeur plus grande que la valeur de chacun de ses fils. Cet arbre binaire est représenté dans un tableau :

- la racine est à l'indice 0,
- l'élément à l'indice i est le père de l'élément à l'indice $2*i+1$ s'il existe, et le père de l'élément de rang $2*i+2$ s'il existe.

Exemple :

6	3	5	2	1	4
---	---	---	---	---	---

est l'équivalent de l'arbre



Mettre un objet dans la file d'attente consiste à ajouter un objet en fin du tableau représentant la pile, puis à le remonter à sa place dans l'arbre.

Retirer un objet de la file d'attente, c'est retirer le premier élément, puis on met le dernier élément du tableau en première position, et on descend cet élément dans l'arbre.

Les comportements de la classe `FileAttentePrH` sont les mêmes que les fonctionnalités de la classe `FileAttente`. Définir la classe `FileAttentePrH`.